Skill - 1

Encapsulation Builder: Analyze the overhead introduced by protocol headers and determine how encapsulation affects network efficiency.

Aim: To capture network packets and understand how Ethernet, IP, and TCP/UDP headers are added during data transmission.

Whenever data travels through a computer network, each protocol layer adds its own header to the data. These headers introduce overhead. Using Wireshark, we will capture real network packets, examine Ethernet, IP and TCP/UDP headers, calculate the total overhead, and determine how efficiently the network carries application data.

Step 1: Install Wireshark

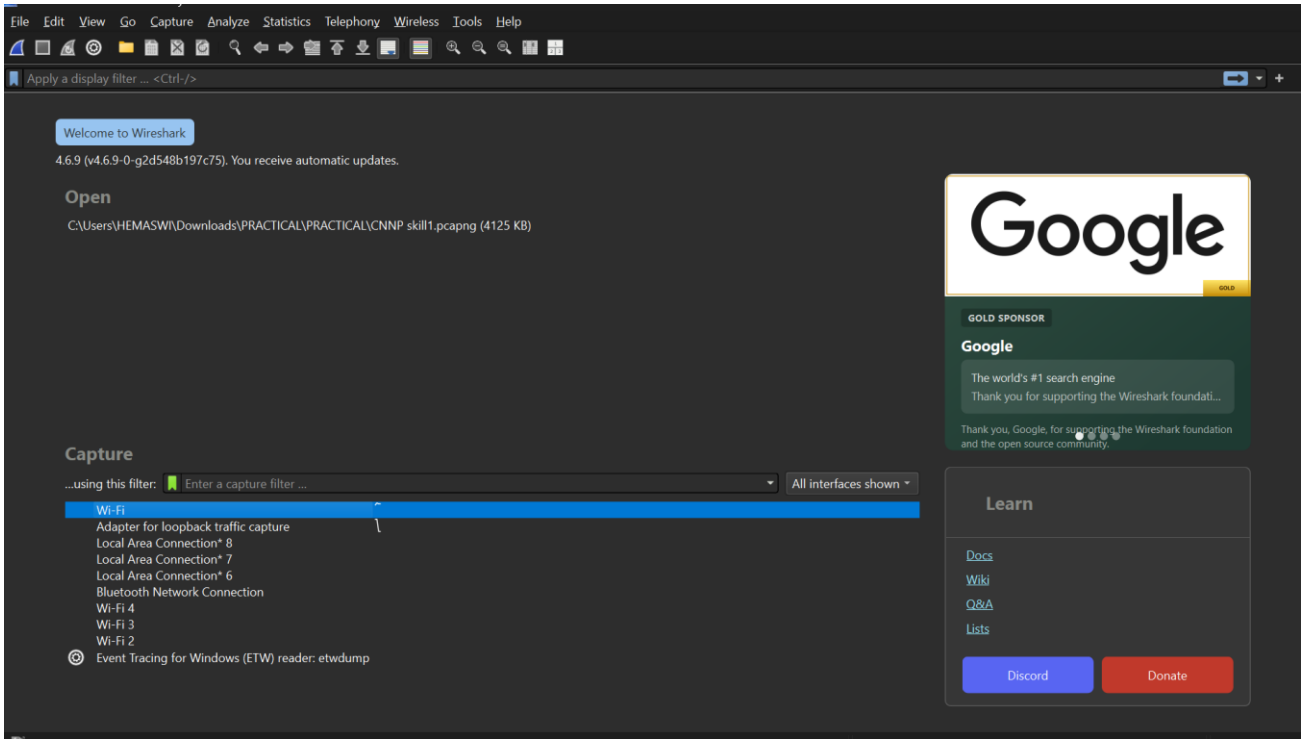
1. Open the browser.
2. Install Wireshark [Wireshark • Go Deep | Download]
3. Install it using the default settings.
4. Open Wireshark.

Step 2: Select your network interface

When Wireshark opens, you will see interfaces such as:

Wi-Fi, Ethernet, Bluetooth

1. If you are connected through Wi-Fi, select Wi-Fi.
2. Double-click the Wi-Fi interface.
3. Packet capture will start.

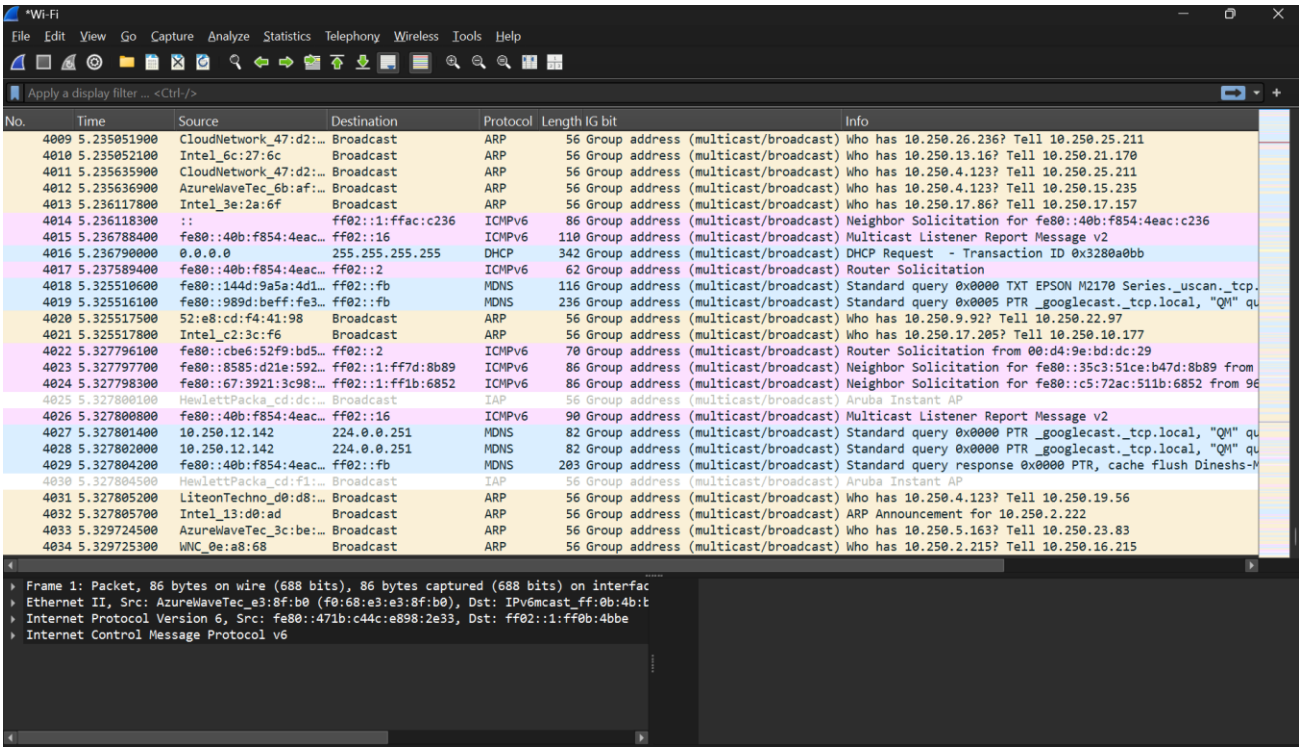


Step 3: Generate network traffic

While Wireshark is capturing:

1. Open a new browser tab.
2. Visit a website such as https://www.google.com.
3. Open two or three different websites.
4. Wait for approximately 30 seconds.
5. Return to Wireshark.
6. Click the red Stop button.

You should now see captured packets.



Step 4: Apply display filters

In the Wireshark filter bar, enter each filter separately and press Enter.

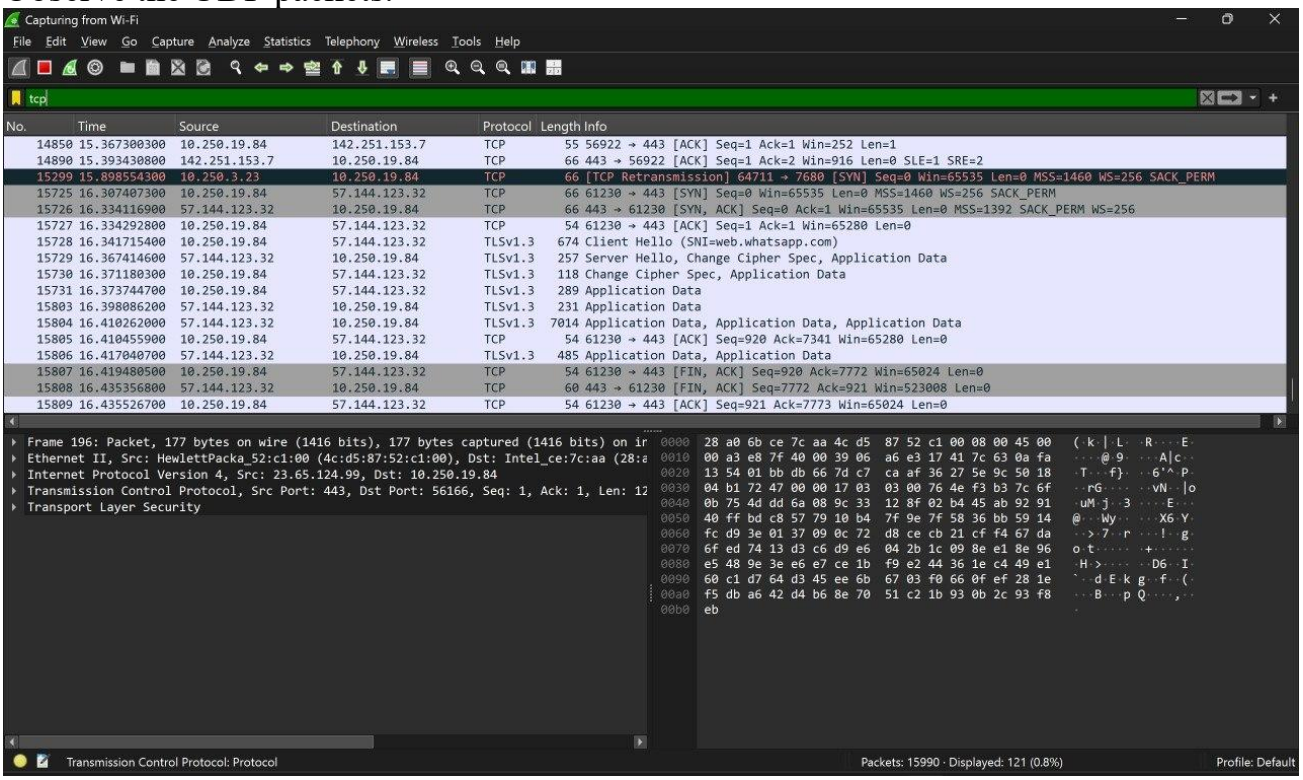
Filter	**Purpose**
tcp	Displays TCP packets
udp	Displays UDP packets
icmp	Displays ICMP packets
tcp.len > 0	Displays TCP packets carrying data

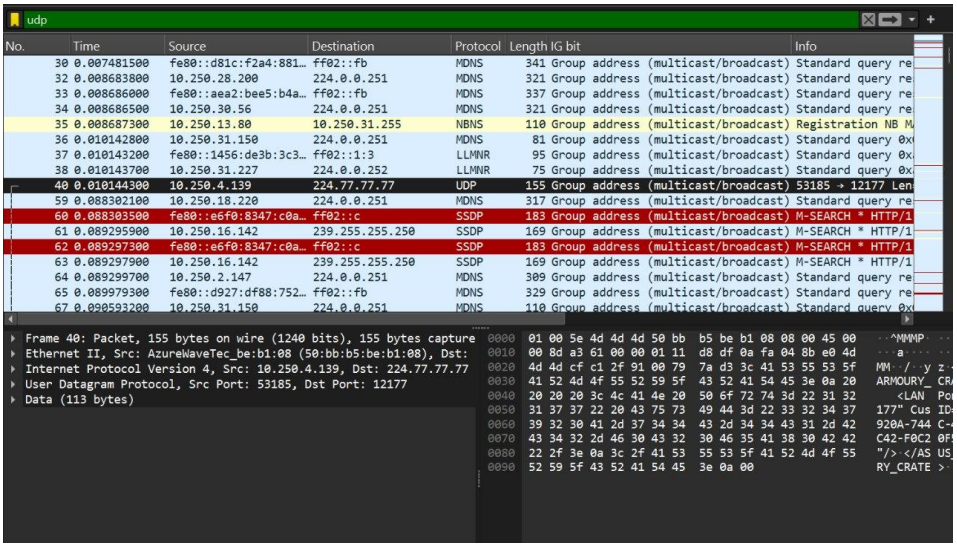
First, enter: TCP

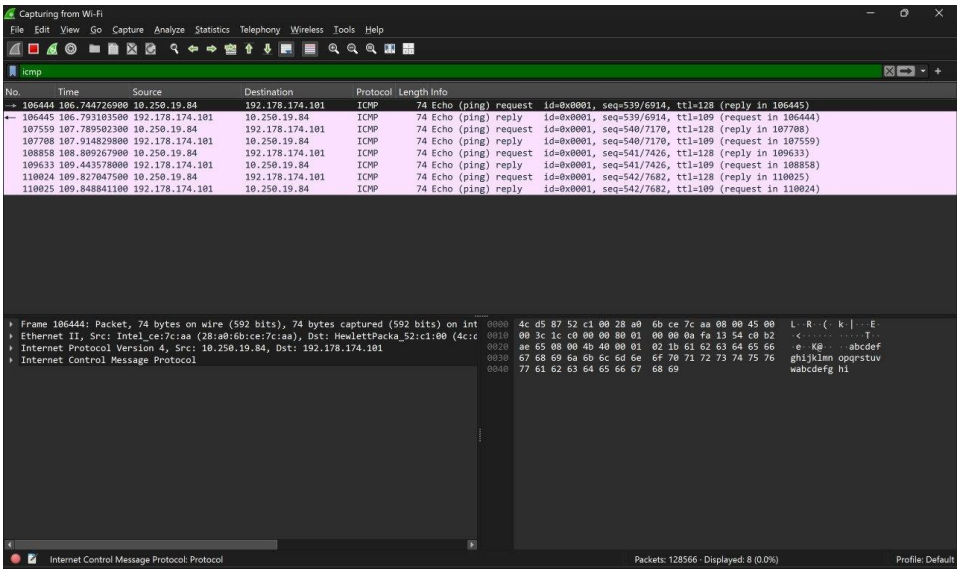
Observe the packets displayed.

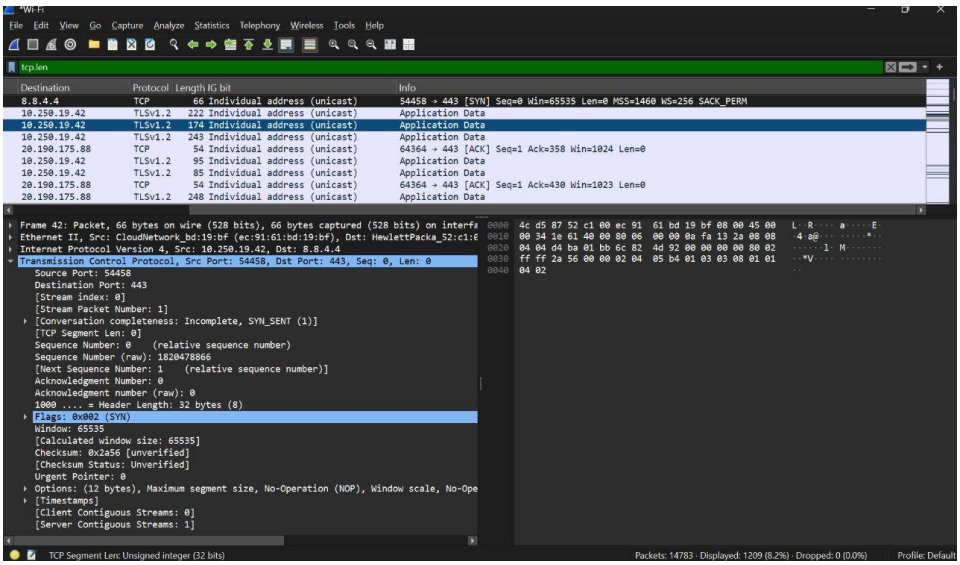
Then enter: UDP

Observe the UDP packets.




udp
No. Time Source Destination Protocol Length IG bit Info
30 0.007481500 fe80::d81c:f2a4:881… ff02::fb MDNS 341 Group address (multicast/broadcast) Standard query re
32 0.008683800 10.250.28.200 224.0.0.251 MDNS 321 Group address (multicast/broadcast) Standard query re
33 0.008686000 fe80::aea2:bee5:b4a… ff02::fb MDNS 337 Group address (multicast/broadcast) Standard query re
34 0.008686500 10.250.30.56 224.0.0.251 MDNS 321 Group address (multicast/broadcast) Standard query re
35 0.008687300 10.250.13.80 10.250.31.255 NBNS 110 Group address (multicast/broadcast) Registration NB M
36 0.010142800 10.250.31.150 224.0.0.251 MDNS 81 Group address (multicast/broadcast) Standard query 0x
37 0.010143200 fe80::1456:de3b:3c3… ff02::1:3 LLMNR 95 Group address (multicast/broadcast) Standard query 0x
38 0.010143700 10.250.31.227 224.0.0.252 LLMNR 75 Group address (multicast/broadcast) Standard query 0x
40 0.010144300 10.250.4.139 224.77.77.77 UDP 155 Group address (multicast/broadcast) 53185 → 12177 Len
59 0.088302100 10.250.18.220 224.0.0.251 MDNS 317 Group address (multicast/broadcast) Standard query re
60 0.088303500 fe80::e6f0:8347:c0a… ff02::c SSDP 183 Group address (multicast/broadcast) M-SEARCH * HTTP/1
61 0.089295900 10.250.16.142 239.255.255.250 SSDP 169 Group address (multicast/broadcast) M-SEARCH * HTTP/1
62 0.089297300 fe80::e6f0:8347:c0a… ff02::c SSDP 183 Group address (multicast/broadcast) M-SEARCH * HTTP/1
63 0.089297900 10.250.16.142 239.255.255.250 SSDP 169 Group address (multicast/broadcast) M-SEARCH * HTTP/1
64 0.089299700 10.250.2.147 224.0.0.251 MDNS 309 Group address (multicast/broadcast) Standard query re
65 0.089979300 fe80::d927:df88:752… ff02::fb MDNS 329 Group address (multicast/broadcast) Standard query re
67 0.090593200 10.250.31.150 224.0.0.251 MDNS 110 Group address (multicast/broadcast) Standard query 0x
Frame 40: Packet, 155 bytes on wire (1240 bits), 155 bytes capture
Ethernet II, Src: AzureWaveTec_be:b1:08 (50:bb:b5:be:b1:08), Dst:
Internet Protocol Version 4, Src: 10.250.4.139, Dst: 224.77.77.77
User Datagram Protocol, Src Port: 53185, Dst Port: 12177
Data (113 bytes)
0000 01 00 5e 4d 4d 4d 50 bb b5 be b1 08 08 00 45 00 ··^MMMP· ····E·
0010 00 8d a3 61 00 00 01 11 d8 df 0a fa 8b e0 4d ···a···· ····M·
0020 4d 4d cf c1 2f 91 00 79 7a d3 3c 41 53 55 53 5f MM··/··y z·<ASUS_
0030 41 52 4d 4f 55 52 59 5f 43 52 41 54 45 3e 0a 20 ARMOURY_ CRATE>·
0040 20 20 20 3c 4c 41 4e 20 50 6f 72 74 3d 22 31 32 <LAN Po
0050 31 37 37 22 20 43 75 73 49 44 3d 22 33 32 34 37 177" Cus ID="3247
0060 39 32 30 41 2d 37 34 34 43 2d 34 34 43 31 2d 42 920A-744 C-44C1-B
0070 43 34 32 2d 46 30 43 32 30 46 35 41 38 30 42 42 C42-F0C2 0F5A80BB
0080 22 2f 3e 0a 3c 2f 41 53 55 53 5f 41 52 4d 4f 55 "/>·</AS USUS_ARMOU
0090 52 59 5f 43 52 41 54 45 3e 0a 00 RY_CRATE >··


Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
icmp
No. Time Source Destination Protocol Length Info
106444 106.744726900 10.250.19.84 192.178.174.101 ICMP 74 Echo (ping) request id=0x0001, seq=539/6914, ttl=128 (reply in 106445)
106445 106.793103500 192.178.174.101 10.250.19.84 ICMP 74 Echo (ping) reply id=0x0001, seq=539/6914, ttl=109 (request in 106444)
107559 107.789502300 10.250.19.84 192.178.174.101 ICMP 74 Echo (ping) request id=0x0001, seq=540/7170, ttl=128 (reply in 107708)
107708 107.914829800 192.178.174.101 10.250.19.84 ICMP 74 Echo (ping) reply id=0x0001, seq=540/7170, ttl=109 (request in 107559)
108858 108.809267900 10.250.19.84 192.178.174.101 ICMP 74 Echo (ping) request id=0x0001, seq=541/7426, ttl=128 (reply in 109633)
109633 109.443578000 192.178.174.101 10.250.19.84 ICMP 74 Echo (ping) reply id=0x0001, seq=541/7426, ttl=109 (request in 108858)
110024 109.827047500 10.250.19.84 192.178.174.101 ICMP 74 Echo (ping) request id=0x0001, seq=542/7682, ttl=128 (reply in 110025)
110025 109.848841100 192.178.174.101 10.250.19.84 ICMP 74 Echo (ping) reply id=0x0001, seq=542/7682, ttl=109 (request in 110024)
Frame 106444: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on int
Ethernet II, Src: Intel_ce:7c:aa (28:a0:6b:ce:7c:aa), Dst: HewlettPacka_52:c1:00 (4c:c
Internet Protocol Version 4, Src: 10.250.19.84, Dst: 192.178.174.101
Internet Control Message Protocol
0000 4c d5 87 52 c1 00 28 a0 6b ce 7c aa 08 00 45 00 L··R··(· k·|···E·
0010 00 3c 1c c0 00 00 80 01 00 00 0a fa 13 54 c0 b2 ·<·····T··
0020 ae 65 08 00 4b 40 00 01 02 1b 61 62 63 64 65 66 ·e··K@·· ··abcdef
0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 ghijklmn opqrstuv
0040 77 61 62 63 64 65 66 67 68 69 wabcdefg hi
Internet Control Message Protocol: Protocol
Packets: 128566 · Displayed: 8 (0.0%)
Profile: Default

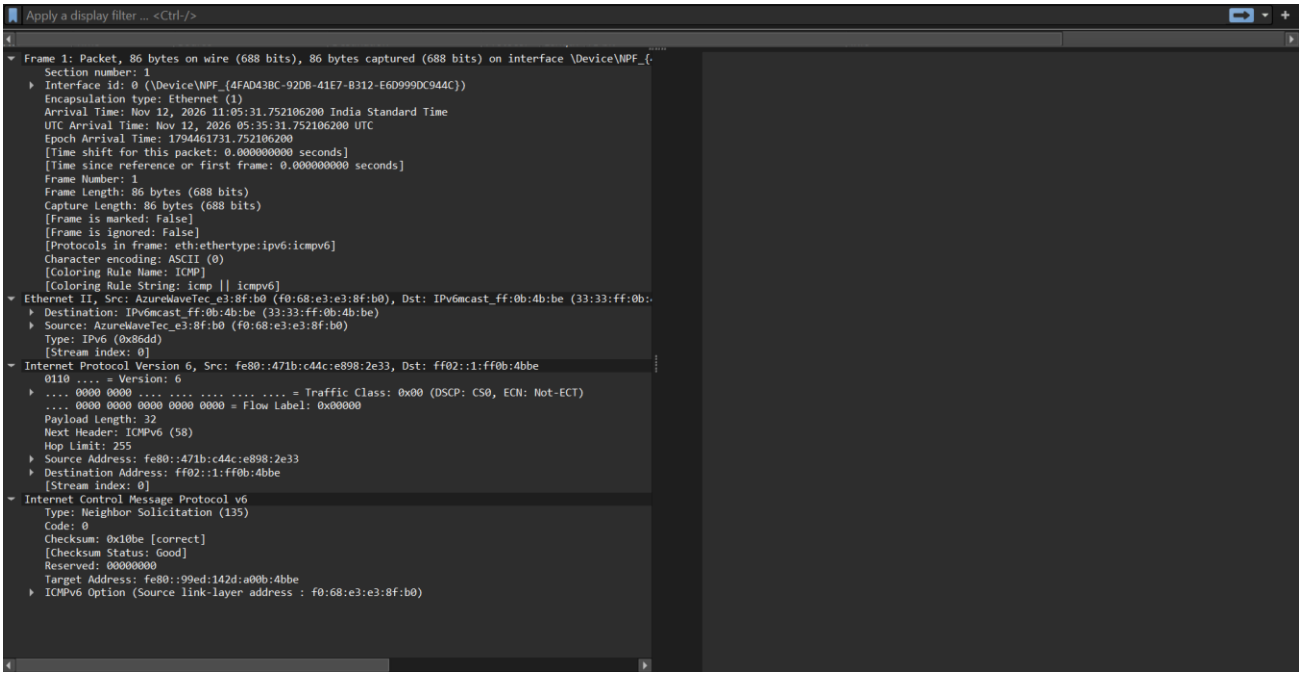

*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.len
Destination Protocol Length IG bit Info
8.8.4.4 TCP 66 Individual address (unicast) 54458 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
10.250.19.42 TLSv1.2 222 Individual address (unicast) Application Data
10.250.19.42 TLSv1.2 174 Individual address (unicast) Application Data
10.250.19.42 TLSv1.2 243 Individual address (unicast) Application Data
20.190.175.88 TCP 54 Individual address (unicast) 64364 → 443 [ACK] Seq=1 Ack=358 Win=1024 Len=0
10.250.19.42 TLSv1.2 95 Individual address (unicast) Application Data
10.250.19.42 TLSv1.2 85 Individual address (unicast) Application Data
20.190.175.88 TCP 54 Individual address (unicast) 64364 → 443 [ACK] Seq=1 Ack=430 Win=1023 Len=0
20.190.175.88 TLSv1.2 248 Individual address (unicast) Application Data
Frame 42: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interfa
Ethernet II, Src: CloudNetwork_bd:19:bf (ec:91:61:bd:19:bf), Dst: HewlettPacka_52:c1:0
Internet Protocol Version 4, Src: 10.250.19.42, Dst: 8.8.4.4
Transmission Control Protocol, Src Port: 54458, Dst Port: 443, Seq: 0, Len: 0
Source Port: 54458
Destination Port: 443
[Stream index: 0]
[Stream Packet Number: 1]
[Conversation completeness: Incomplete, SYN_SENT (1)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 1820478866
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 .... = Header Length: 32 bytes (8)
Flags: 0x002 (SYN)
Window: 65535
[Calculated window size: 65535]
Checksum: 0x2a56 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Ope
[Timestamps]
[Client Contiguous Streams: 0]
[Server Contiguous Streams: 1]
0000 4c d5 87 52 c1 00 ec 91 61 bd 19 bf 08 00 45 00 L··R···· a·····E·
0010 00 34 1e 61 40 00 80 06 00 00 0a fa 13 2a 08 08 ·4·a@··· ·····*··
0020 04 04 d4 ba 01 bb 6c 82 4d 92 00 00 00 00 80 02 ····· l· M·····
0030 ff ff 2a 56 00 00 02 04 05 b4 01 03 03 08 01 01 ··*V···· ····
0040 04 02 ··
TCP Segment Len: Unsigned integer (32 bits)
Packets: 14783 · Displayed: 1209 (8.2%) · Dropped: 0 (0.0%)
Profile: Default

Step 5: Inspect packet headers

Select any TCP packet from the packet list.

In the packet details pane, expand these sections:

1. Frame
2. Ethernet II
3. Internet Protocol Version 4
4. Transmission Control Protocol



Record the following values:

Parameter	Value
Frame length	86 bytes
Ethernet Header	14 bytes
Source MAC Address	ec:91:61:bd:19:bf
Destination MAC Address	33:33:ff:0b:4b:be
IPv4 Header Length	20 bytes
IPv4 Total Length	52 bytes
TCP Header Length	32 bytes
TCP Segment Length	0 bytes

Repeat the process for one UDP packet. Observe the UDP header and its length field.

Step 6: Calculate header overhead

For a basic Ethernet + IPv4 + TCP packet without TCP options:

- Ethernet header = 14 bytes
- IPv4 header = 20 bytes
- TCP header = 32 bytes
- Application data = 1500 bytes
- FCS = 4 bytes, if included in the frame representation

Total size including FCS:

$$14 + 20 + 32 + 1500 + 4 = 1570 bytes$$

Total overhead:

$$14 + 20 + 32 + 4 = 70bytes$$

Payload efficiency:

$$\frac{1500}{1570} \times 100 = 95.54\%$$

Step 7:
Fill in your observation table

Record values from your own Wireshark capture.

Parameter	**TCP Packet**	**UDP Packet**
Frame Length	66 bytes	108 bytes
Ethernet Header Length	14 bytes	14 bytes
IP Header Length	20 bytes	20 bytes
Transport Header Length	32 bytes	8 bytes
IP Total Length	52 bytes	94 bytes
Transport Header Length	32 bytes	74 bytes
Payload Length	0 bytes	66 bytes

Answer these questions

1. What is encapsulation?
    - Encapsulation is the process of adding headers (and sometimes trailers) to data as it passes through different layers of a network.
2. What is the purpose of the Ethernet header?
    - The Ethernet header contains source and destination MAC addresses and the type of protocol used. It helps deliver data within a local network.
3. What information is stored in the IP header?
    - The IP header stores source IP address, destination IP address, packet length, TTL, and protocol information.
4. What is the difference between TCP and UDP headers?
    - TCP: Contains source port, destination port, sequence number, acknowledgment number, and flags.
    - UDP: Contains source port, destination port, length, and checksum.
    - TCP is reliable, whereas UDP is faster and connectionless.
5. Why does each protocol add a header?
    - Each protocol adds a header to provide necessary information for identifying, routing, delivering, and processing data.
6. What is packet overhead?
    - Packet overhead is the extra data added by network protocols in the form of headers and trailers, excluding the actual application data.
7. How does header overhead affect network efficiency?
    - Higher header overhead reduces network efficiency because more bandwidth is used for control information instead of actual data.